Technical Report

Abhishek Manjunath
Department of Electrical Engineering
April 10, 2026

Abstract

We investigate learned decoding of a rate-$1/2$, constraint-length-7 convolutional code under periodic sinusoidal interference, motivated by industrial IoT channels. Three classical baselines are first evaluated: mismatched Viterbi (B1), oracle Viterbi (B2), and FFT-based interference cancellation (B5). At SNR $= 5$ dB and INR $= 5$ dB, B1 saturates at BLER $= 1.0$, B2 achieves 10^{-4}, and B5 achieves 0.35. An end-to-end BiGRU decoder (N1) fails to learn the task — an information bottleneck prevents implicit trellis search — and exceeds the 2.1 MFLOP compute constraint by $3.17\times$. A neural branch metric estimator (N2) resolves this by keeping Viterbi intact and replacing only the branch metric computation with a small BiGRU ($h = 16$, 913,664 MACs). N2 (seed 32) achieves BLER $= 2.5 \times 10^{-3}$ at the same operating point, a gain exceeding five SNR dB over B1 and ≈ 1.2 dB from the oracle by log-linear interpolation.

I. Introduction

Classical convolutional decoding via the Viterbi algorithm assumes a precisely known channel model. In AWGN-only environments, mismatched branch metrics computed under this assumption achieve near-optimal block-error-rate (BLER) performance. However, in industrial and IoT settings, narrowband sinusoidal interference from motor drives, power converters, and switching regulators regularly coexists with thermal noise [1], [2]. A standard Viterbi decoder that ignores such interference treats the tonal energy as additional noise, causing severe BLER degradation at moderate interference-to-noise ratios (INR).

This report investigates the following question: *Can a computationally constrained learned component, inserted into an otherwise classical Viterbi decoder, recover most of the performance lost to sinusoidal interference?*

We pursue this question in three phases:

1) **Phase 1 — Classical Baselines.** We implement and evaluate the mismatched Viterbi (B1), the oracle Viterbi with perfect interference knowledge (B2), and an FFT-based interference cancellation baseline (B5). These establish the performance floor, ceiling, and a signal-processing mid-point.
2) **Phase 2a — End-to-End Neural Decoder (Negative Result).** A bidirectional GRU is trained to map raw received samples directly to information bits. The architecture fails to learn meaningful decoding, plateauing near random-guessing accuracy. Analysis reveals two root causes: an information bottleneck at the pooling layer and the intractability of implicitly representing a 64-state Viterbi search within a small network. The model also violates the 2.1 MFLOP compute constraint.
3) **Phase 2b — Neural Branch Metric Estimator.** Motivated by the Phase 2a failure, we decompose the decoding task: a small BiGRU learns local per-step branch metrics capturing interference effects; the Viterbi algorithm performs the global trellis search using those metrics. This architecture succeeds, achieving BLER $= 2.5 \times 10^{-3}$ at SNR $= 5$ dB, INR $= 5$ dB, well within the compute budget.

The key thesis is that *problem decomposition* — assigning local channel estimation to a neural network and global sequence search to a classical algorithm — is the critical design principle enabling efficient and effective decoding under structured interference.

II. System Model

A Channel

The discrete-time channel model is

$$r[t] = x[t] + n[t] + i[t], \quad t = 0, 1, \ldots, N-1, \tag{1}$$

where $x[t] \in \{-1, +1\}$ is a BPSK-modulated coded symbol, $n[t] \sim \mathcal{N}(0, \sigma^2)$ is additive white Gaussian noise, and $i[t]$ is a deterministic sinusoidal interferer:

$$i[t] = A \sin\left(\frac{2\pi t}{P} + \varphi\right). \tag{2}$$

The amplitude A is derived from the interference-to-noise ratio (INR):

$$A = \sqrt{2 \cdot 10^{\mathrm{INR_{dB}}/10} \cdot \sigma^2}, \tag{3}$$

which ensures that the average sinusoidal power equals $10^{\mathrm{INR_{dB}}/10} \cdot \sigma^2$. The period P is drawn uniformly from $\{8, 9, \ldots, 32\}$ samples, the phase φ is drawn uniformly from $[0, 2\pi)$, and the noise variance is $\sigma^2 = 10^{-\mathrm{SNR_{dB}}/10}$ (with unit symbol energy $E_s = 1$).

B Trellis Code

The encoder is the NASA standard rate-$1/2$, constraint-length-7 convolutional code with octal generator polynomials $G_1 = 171_8 = (1111001)_2$ and $G_2 = 133_8 = (1011011)_2$ [3]. The constraint length $K_c = 7$ gives memory $m = K_c - 1 = 6$ and $S = 2^m = 64$ trellis states. The free distance is $d_{\mathrm{free}} = 10$.

Encoding processes $K = 256$ information bits. Six tail (zero) bits are appended to terminate the trellis back to state 0, giving $K + m = 262$ total input bits and $N = 2 \times 262 = 524$ coded bits per block. The effective code rate is $R_{\mathrm{eff}} = K/N = 256/524 \approx 0.489$.

C System Parameters

System parameters are given in Table I

TABLE I: Fixed system parameters

Parameter	Value
Modulation	BPSK, $x[t] \in \{-1, +1\}$
Information bits	$K = 256$
Constraint length	$K_c = 7$ (memory $m = 6$)
Trellis states	$S = 64$
Tail bits	$m = 6$ (trellis termination)
Total input bits	$K + m = 262$
Coded block length	$N = 524$
Nominal code rate	$R = 1/2$
Effective code rate	$R_{\mathrm{eff}} = 256/524 \approx 0.489$
Free distance	$d_{\mathrm{free}} = 10$
SNR range	[0, 10] dB
INR range	[−5, 15] dB
Interference period	$P \in [8, 32]$ samples
Interference phase	$\varphi \in [0, 2\pi)$
Eval trials (standard)	10,000 blocks
Target BLER	10^{-3}

D Compute Budget

A compute constraint of 2.1×10^6 multiply-accumulate operations (MACs) per block is enforced for all methods. All profiling uses a consistent MAC-based convention throughout: one MAC counts one multiply-add pair (e.g., a squared difference $(y - x)^2$ is one MAC, not two separate operations). The budget is anchored to the cost of the Viterbi algorithm profiled under this convention.

The Viterbi add-compare-select (ACS) loop uses $N = 512$ coded bits (the $K = 256$ information bits at rate $1/2$, excluding tail sections for this budget calculation). There are $T = N/2 = 256$ trellis steps, $S = 64$ destination states, 2 incoming branches per state, and 2 output symbols per branch, each requiring 1 MAC:

$$C_{\text{Viterbi}} = T \times S \times 2 \times 2 = 256 \times 64 \times 4 = 65{,}536 \text{ MACs}. \tag{4}$$

The 2.1 MFLOP budget therefore allows roughly $32\times$ the cost of the Viterbi ACS loop, providing generous headroom for a neural component while remaining tractable for edge deployment. All methods reported in this work are verified to satisfy this constraint.

III. Phase 1: Classical Baselines

A Mismatched Viterbi (B1)

The mismatched Viterbi decoder assumes an AWGN-only channel. At trellis step t, the branch metric for the transition $s \to s'$ producing BPSK hypothesis $\mathbf{x}_{s\to s'} \in \{-1, +1\}^2$ is

$$\gamma_{\text{B1}}(s \to s', k) = -\frac{\|\mathbf{y}[k] - \mathbf{x}_{s\to s'}\|^2}{2\sigma^2}, \tag{5}$$

where $\mathbf{y}[k] = (r[2k],\, r[2k+1])^T$ denotes the received pair at trellis step k (distinct from the scalar sample $r[t]$ in Eq. (1)). Because the interference term $i[t]$ is not modeled, B1 treats sinusoidal energy as noise, causing BLER saturation even at moderate INR.

B Oracle Viterbi (B2)

The oracle Viterbi has full knowledge of A, P, and φ and subtracts the interference before computing branch metrics:

$$\gamma_{\text{B2}}(s \to s', k) = -\frac{\|\mathbf{y}[k] - \mathbf{x}_{s\to s'} - \mathbf{i}[k]\|^2}{2\sigma^2}, \tag{6}$$

where $\mathbf{i}[k] = (i[2k],\, i[2k+1])^T$ is the known interference pair at trellis step k. B2 represents the performance ceiling: it achieves optimal ML decoding under the true channel model and serves as the upper bound for all methods.

C Interference Cancellation + Viterbi (B5)

B5 estimates the interference parameters from the received signal and applies a subtraction prior to mismatched Viterbi decoding:

$$\tilde{\mathbf{y}}[k] = \mathbf{y}[k] - \hat{A} \begin{pmatrix} \sin(2\pi(2k)/\hat{P} + \hat{\varphi}) \\ \sin(2\pi(2k{+}1)/\hat{P} + \hat{\varphi}) \end{pmatrix}, \tag{7}$$

followed by the B1 branch metric on $\tilde{\mathbf{y}}[k]$.

The estimation pipeline proceeds as follows:

1) Compute the periodogram of $r[t]$ via FFT.
2) Identify the top-5 magnitude peaks whose frequencies fall within the valid range $[1/P_{\max}, 1/P_{\min}]$.
3) For each candidate frequency $\omega_k = 2\pi f_k$, fit $r[t] \approx a_k \cos(\omega_k t) + b_k \sin(\omega_k t) + c$ via least squares: $\hat{\mathbf{c}}_k = (\mathbf{A}_k^T \mathbf{A}_k)^{-1} \mathbf{A}_k^T \mathbf{r}$.
4) Recover amplitude and phase: $\hat{A}_k = \sqrt{a_k^2 + b_k^2}$, $\hat{\varphi}_k = \operatorname{atan2}(a_k, b_k)$.
5) Select the candidate minimizing the residual $\sum_t (r[t] - \hat{A}_k \sin(\omega_k t + \hat{\varphi}_k))^2$.

B5 demonstrates the limit of signal-processing-only interference mitigation.

D Compute Cost

Table II summarizes the compute cost of all three classical baselines what was calculated using `thop`. All consume less than 5% of the 2.1 MFLOP budget, leaving substantial headroom for any neural component.

TABLE II: Phase 1 compute cost (per block, 10,000-trial latency median)

Method	FLOPs	Latency (ms)	% of 2.1M budget
B1 Mismatched Viterbi	65,536	0.23	3.1%
B2 Oracle Viterbi	65,536	0.27	3.1%
B5 IC + Viterbi	94,720	0.93	4.5%

E Results

E1 BLER vs. SNR (INR = 5 dB)

Table III and Fig. 1 show the BLER vs. SNR performance of all three baselines at INR $= 5$ dB.

TABLE III: BLER vs. SNR at INR $= 5$ dB (10,000 trials per point)

SNR (dB)	B1	B2	B5
0	1.000	9.46×10^{-1}	9.93×10^{-1}
2	1.000	1.50×10^{-1}	7.69×10^{-1}
4	1.000	1.60×10^{-3}	4.79×10^{-1}
5	1.000	1.00×10^{-4}	3.50×10^{-1}
6	9.79×10^{-1}	$<10^{-4}$	2.13×10^{-1}
8	1.02×10^{-1}	$<10^{-4}$	2.65×10^{-2}
10	1.00×10^{-4}	$<10^{-4}$	1.00×10^{-4}

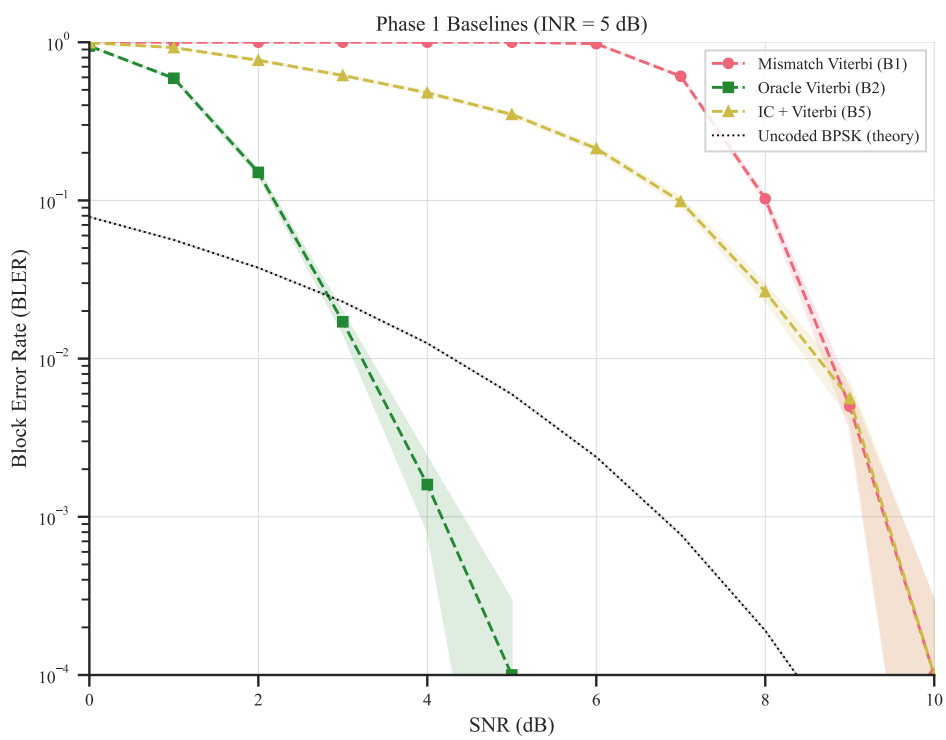


Fig. 1: BLER vs. SNR at INR $= 5$ dB for B1, B2, and B5. See Table III for numerical values.

E2 BLER vs. INR (SNR = 5 dB)

Table IV and Fig. 2 show the BLER vs. INR performance at SNR $= 5$ dB across the full INR range.

The key observation is that at SNR $= 5$ dB the interference becomes catastrophic for B1 at INR ≈ 5 dB, and B5's estimation error limits its benefit beyond INR ≈ 0 dB. The ≈ 17 dB gap between B1 and B2 at INR $= 5$ dB defines the opportunity for a learned approach.

TABLE IV: BLER vs. INR at SNR $= 5$ dB (10,000 trials per point)

INR (dB)	B1	B2	B5
-5	1.7×10^{-3}	1.0×10^{-4}	3.0×10^{-4}
0	1.14×10^{-1}	1.0×10^{-4}	2.49×10^{-2}
5	1.000	1.0×10^{-4}	3.50×10^{-1}
10	1.000	1.0×10^{-4}	6.73×10^{-1}
15	1.000	1.0×10^{-4}	8.20×10^{-1}

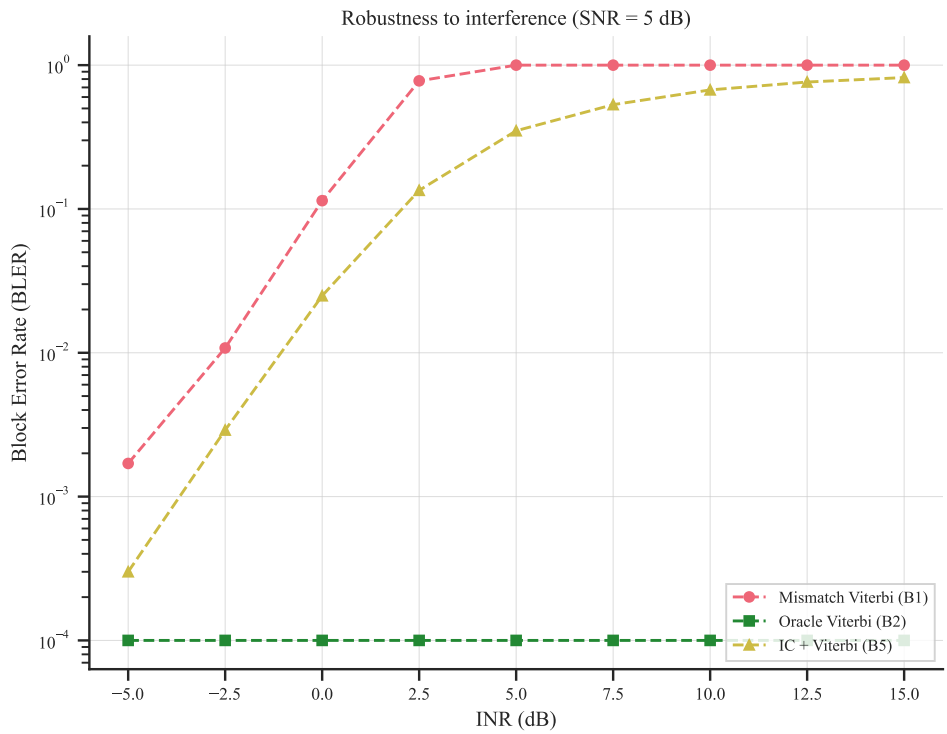


Fig. 2: BLER vs. INR at SNR $= 5$ dB for B1, B2, and B5. B2 is invariant to INR; B1 and B5 degrade with increasing interference. See Table IV for numerical values.

IV. Phase 2a: End-to-End Neural Decoder

A Architecture

Motivated by prior work on RNN-based channel decoders [5], [6], we implemented a bidirectional GRU (BiGRU) end-to-end decoder, denoted N1. The architecture is:

- **Input:** $(B, 524, 1)$ — raw received samples, one per time step.
- **BiGRU:** hidden size $h = 32$ per direction, bidirectional, 1 layer. Output: $(B, 524, 64)$ all-timestep hidden states.
- **Pooling:** Final hidden states from both directions concatenated: $(B, 64)$.
- **Output head:** Linear$(64, 256) \rightarrow (B, 256)$ logits.

Total parameter count: $\approx 23{,}000$. Three pooling strategies were evaluated: mean pooling over all 524 time steps ($\approx 52.1\%$ bit accuracy), final-hidden-state pooling ($\approx 53.5\%$), and attention-weighted pooling ($\approx 53.8\%$).

B Training Configuration

Training used binary cross-entropy with logits (BCEWithLogitsLoss), the Adam optimizer at lr $= 10^{-3}$ with StepLR decay, batch size 128, and on-the-fly batch generation at SNR $\in [0, 10]$ dB, INR $\in$

$[-5, 15]$ dB, $P \in [8, 32]$. Validation BLER was measured every 5 epochs on 1,000 blocks at SNR $= 5$ dB, INR $= 5$ dB; early stopping used patience 20.

C Analytical Compute Budget

The BiGRU with $h = 32$, input dimension $d = 1$, applied to the full $T = 524$-sample sequence incurs the following FLOPs per block. Each GRU gate (input, reset, update) requires $h \times (h + d) = 32 \times 33$ multiply-accumulate (MAC) operations; there are 3 gates, giving $6 \times h \times (h+d)$ MACs per time step per direction:

$$\begin{aligned} C_{\text{GRU,dir}} &= T \times 6 \times h \times (h + d) \\ &= 524 \times 6 \times 32 \times 33 = 3{,}320{,}064. \end{aligned} \tag{8}$$

Both directions together, plus the linear output head:

$$\begin{aligned} C_{\text{N1}} &= 2 \times C_{\text{GRU,dir}} + 64 \times 256 \\ &= 6{,}640{,}128 + 16{,}384 \approx 6.66 \text{ MFLOP}. \end{aligned} \tag{9}$$

This is $3.17\times$ **the** 2.1 **MFLOP constraint**, making N1 inadmissible under the compute budget even if it had learned to decode.

D Experimental Results

Across all training configurations and pooling strategies, the model plateaued at approximately 53.5% bit accuracy, only marginally above the 50% random-guessing baseline. BLER remained at 1.0 throughout all training runs — no block was ever decoded correctly.

An overfitting sanity check (4 fixed blocks, 500 epochs) confirmed that the architecture has sufficient capacity to memorize small datasets (bit accuracy 100%, loss < 0.003), ruling out implementation errors.

E Failure Analysis

Two structural problems preclude N1 from succeeding:

a) Information bottleneck.

The pooling operation compresses the full 524-step sequence into a 64-dimensional vector, from which 256 information bits must be recovered. The mutual information of a BPSK convolutional codeword transmitted over this channel is far greater than what a 64-dim vector can represent; the bottleneck discards the structured redundancy that Viterbi exploits through its trellis search.

b) Implicit trellis search.

Viterbi exploits the Markov structure of the code via dynamic programming over 64 states across 262 trellis sections. For N1 to succeed, the GRU hidden state would need to implicitly represent this exponential-space search — a combinatorially intractable mapping for a network with $\approx 23{,}000$ parameters.

c) Design lesson.

The failure of N1 motivates a strict decomposition: assign the problem of *local channel estimation* (what is the likely interference at time t?) to the neural component, and the problem of *global sequence search* (what path through the trellis best explains the full received sequence?) to the Viterbi algorithm. This decomposition is the foundation of Phase 2b.

V. Phase 2b: Neural Branch Metric Estimator

A Motivation

The Viterbi algorithm decomposes the MAP sequence detection problem into a dynamic program over branch metrics $\gamma(s \to s', t)$. In the oracle decoder (B2), these metrics are exact log-likelihoods under the true channel. The key insight is that the oracle metric requires knowledge of A, P, and φ — three

continuously-valued parameters that change per block — whereas Viterbi itself is a deterministic algorithm that requires only a metric table.

N2 replaces the closed-form oracle metric computation with a BiGRU that *learns* to predict those metrics from the received signal, without ever explicitly estimating A, P, or φ.

B Architecture

The neural branch metric estimator processes received signals grouped into pairs, aligned with the trellis time axis. The full architecture is as follows:

- **Input:** $(B, T, 2)$ with $T = K + m = 262$. Consecutive received samples are grouped into pairs $\mathbf{y}[k] = (r[2k],\, r[2k+1])^T$.
- **BiGRU:** hidden size $h = 16$ per direction, 1 layer. Output: $(B, T, 32)$ — both-direction hidden states concatenated.
- **Linear head:** Linear$(32, 4) \to (B, T, 4)$ — one score per BPSK output pair at each trellis step.

The four BPSK output pairs correspond to all combinations $\mathbf{h}_j \in \{-1, +1\}^2$: $\mathbf{h}_0 = (-1, -1)$, $\mathbf{h}_1 = (-1, +1)$, $\mathbf{h}_2 = (+1, -1)$, $\mathbf{h}_3 = (+1, +1)$. During Viterbi, the score for transition $s \to s'$ at step t is looked up as $\hat{\gamma}[t, j(s \to s')]$, where $j(s \to s')$ is the precomputed index of the BPSK pair produced by that transition.

Total parameter count: $\approx 1{,}860$ — approximately $12.5\times$ fewer parameters than N1.

C Oracle Training Targets and Loss Function

For each training sample, the true interference $\mathbf{i}[t]$ is known (generated on-the-fly during training). The oracle branch metric for hypothesis $\mathbf{h}_j$ at step t is the negative squared Euclidean distance:

$$\gamma_j^*(k) = -\left\|\mathbf{y}[k] - \mathbf{h}_j - \mathbf{i}[k]\right\|^2. \tag{10}$$

This is the unnormalized oracle log-likelihood; the $1/(2\sigma^2)$ scaling is applied at inference time after the NN forward pass.

The network is trained with mean squared error (MSE) loss:

$$\mathcal{L} = \frac{1}{B \cdot T \cdot 4} \sum_{b,t,j} \left(\hat{\gamma}_{b,j}(t) - \gamma_{b,j}^*(t)\right)^2. \tag{11}$$

MSE on the oracle metrics (rather than cross-entropy on information bits) is the natural loss for this regression task and is the approach adopted by ViterbiNet [4] and related hybrid model-based/data-driven architectures.

D Training Configuration

The full set of training hyperparameters is listed in Table V.

Two random seeds were trained with different oracle metric normalizations:

- **Seed 42 (normalized targets):** Training targets include the $1/(2\sigma^2)$ factor; the NN is trained to predict the full log-likelihood. No post-processing is applied at inference.
- **Seed 32 (external normalization):** Training targets are the raw unnormalized distances $-\left\|\mathbf{y}[k] - \mathbf{h}_j - \mathbf{i}[k]\right\|^2$. At inference, the NN output is divided by $2\sigma^2$ before passing to Viterbi. This decouples the NN's learning task (estimating interference-aware distances) from the noise-level scaling, which is known at inference time.

E Viterbi Integration

The N2 decode pipeline proceeds in two stages:

1) **NN forward pass:** A single forward pass through the BiGRU produces the branch metric table $\hat{\mathbf{\Gamma}} \in \mathbb{R}^{T \times 4}$ for the entire block. For seed 32, this is divided element-wise by $2\sigma^2$.

TABLE V: N2 training hyperparameters

Parameter	Value
Loss	MSE on oracle branch metrics
Optimizer	Adam
Learning rate	10^{-3}, StepLR decay
Batch size	256
Batches per epoch	200
Max epochs	200
Early stopping patience	20 epochs
Training SNR	Uniform $\in [0, 10]$ dB
Training INR	Uniform $\in [-5, 15]$ dB
Training P	Uniform integer $\in [8, 32]$
Training φ	Uniform $\in [0, 2\pi)$
Validation SNR / INR	5 dB / 5 dB
Validation blocks	1,000 per check
Validation interval	Every 5 epochs

2) **Viterbi ACS:** Standard add-compare-select is applied using $\hat{\mathbf{\Gamma}}$, with the precomputed transition-to-output-index table $j(s \to s')$. Traceback recovers the $K = 256$ decoded information bits.

The NN forward pass and the Viterbi step are fully separable, enabling efficient implementation. No per-step NN calls are required.

F Compute Cost

The analytical FLOPs for N2 with $h = 16$, $d = 2$, $T = 262$ are:

$$C_{\text{GRU,dir}} = 262 \times 6 \times 16 \times (16 + 2) = 453{,}888, \tag{12}$$

$$C_{\text{N2}} = 2 \times 453{,}888 + 262 \times 32 \times 4 = 907{,}776 + 33{,}536 \approx 941{,}312 \text{ FLOPs}. \tag{13}$$

Profiling via `thop` measured 913,664 FLOPs (the small discrepancy reflects gate-level counting conventions in PyTorch). Table VI compares N2 with the classical baselines.

TABLE VI: Phase 2b compute cost (per block); B1/B2/B5 repeated for comparison

Method	FLOPs	Latency (ms)	% of 2.1M budget
B1 Mismatched Viterbi	65,536	0.30	3.1%
B2 Oracle Viterbi	65,536	0.35	3.1%
B5 IC + Viterbi	94,720	1.24	4.5%
N2 (seed 32)	913,664	1981.9	43.5%
N2 (seed 42)	913,664	1614.2	43.5%

N2 uses 43.5% of the 2.1 MFLOP budget, well within the constraint. The high wall-clock latency reflects unoptimized PyTorch inference on CPU without batching. The models however, were trained on GPU and the latency was not captured; FLOP count is the relevant metric for comparing architectures.

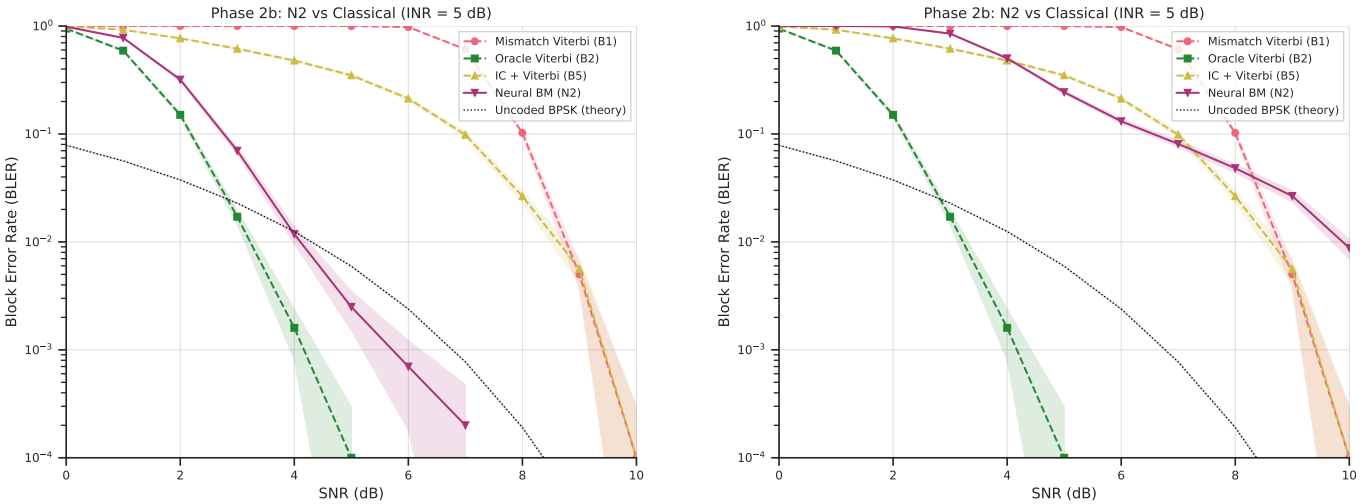
G Results: BLER vs. SNR

Table VII and Fig. 3 show BLER vs. SNR at INR = 5 dB for both N2 seeds alongside the three baselines.

Seed 32 achieves BLER $= 2.5 \times 10^{-3}$ at SNR = 5 dB, INR = 5 dB, representing a gain of more than five SNR dB over B1 (which saturates at BLER = 1.0). To quantify the gap to the oracle, log-linear interpolation of the B2 curve identifies SNR ≈ 3.81 dB as the point where B2 achieves the matched BLER of 2.5×10^{-3}, giving a gap of $5.00 - 3.81 \approx 1.2$ dB relative to B2. Seed 42 achieves BLER $= 0.244$ at the same operating point — a significant improvement over B1 and B5, but substantially weaker than seed 32.

TABLE VII: BLER vs. SNR at INR $= 5$ dB, both N2 seeds (10,000 trials)

SNR (dB)	B1	B2	B5	N2 (s32)	N2 (s42)
0	1.000	9.46×10^{-1}	9.93×10^{-1}	9.86×10^{-1}	1.000
2	1.000	1.50×10^{-1}	7.69×10^{-1}	3.19×10^{-1}	9.93×10^{-1}
4	1.000	1.60×10^{-3}	4.79×10^{-1}	1.18×10^{-2}	5.03×10^{-1}
5	1.000	1.00×10^{-4}	3.50×10^{-1}	2.50×10^{-3}	2.44×10^{-1}
6	9.79×10^{-1}	$< 10^{-4}$	2.13×10^{-1}	7.00×10^{-4}	1.31×10^{-1}
7	6.10×10^{-1}	$< 10^{-4}$	9.84×10^{-2}	2.00×10^{-4}	8.09×10^{-2}
8	1.02×10^{-1}	$< 10^{-4}$	2.65×10^{-2}	$< 10^{-4}$	4.82×10^{-2}
10	1.00×10^{-4}	$< 10^{-4}$	1.00×10^{-4}	$< 10^{-4}$	8.80×10^{-3}



(a) Seed 32 (external σ^2 normalization)

(b) Seed 42 (normalized training targets)

Fig. 3: BLER vs. SNR at INR $= 5$ dB for N2 (both seeds) and baselines. Numerical values in Table VII.

H Results: BLER vs. INR

Table VIII and Fig. 4 show the BLER vs. INR performance at SNR $= 5$ dB for both N2 seeds and the baselines.

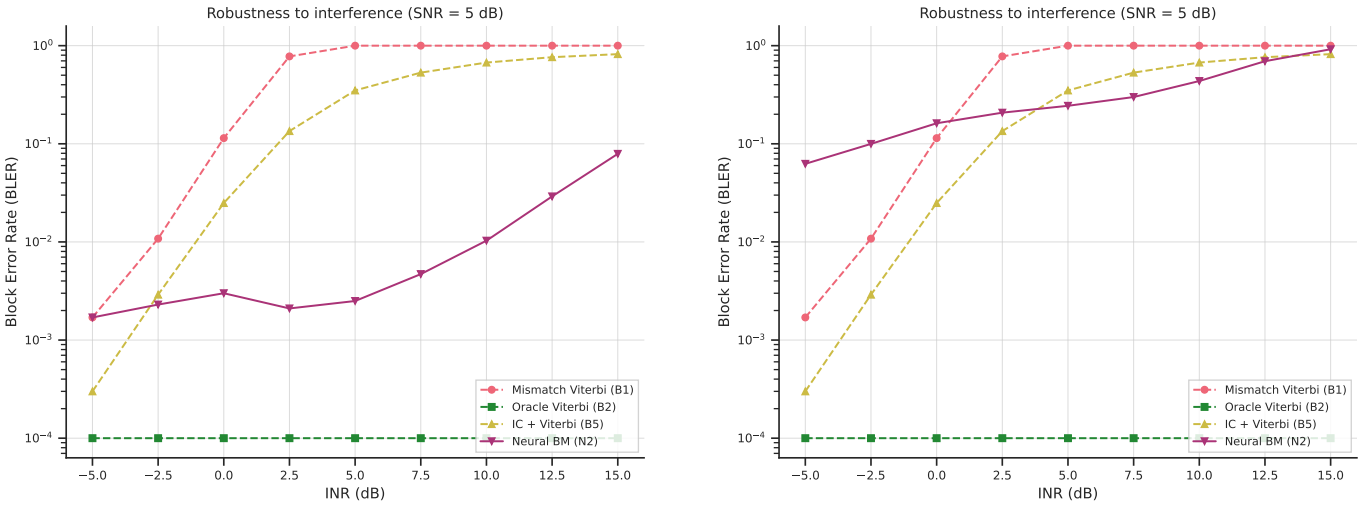
TABLE VIII: BLER vs. INR at SNR $= 5$ dB, both N2 seeds (10,000 trials)

INR (dB)	B1	B2	B5	N2 (s32)	N2 (s42)
-5	1.70×10^{-3}	1.0×10^{-4}	3.0×10^{-4}	1.70×10^{-3}	6.25×10^{-2}
0	1.14×10^{-1}	1.0×10^{-4}	2.49×10^{-2}	3.00×10^{-3}	1.62×10^{-1}
5	1.000	1.0×10^{-4}	3.50×10^{-1}	2.50×10^{-3}	2.44×10^{-1}
10	1.000	1.0×10^{-4}	6.73×10^{-1}	1.03×10^{-2}	4.36×10^{-1}
15	1.000	1.0×10^{-4}	8.20×10^{-1}	7.90×10^{-2}	9.19×10^{-1}

Seed 32 shows *graceful degradation* with increasing INR, maintaining BLER below 10^{-2} up to INR $=$ 10 dB. B1 has already saturated at BLER $= 1$ above INR ≈ 5 dB. At INR $= -5$ dB (almost no interference), seed 32 matches B1 exactly, confirming that the neural branch metrics recover the standard Euclidean distance metric in the interference-free limit.

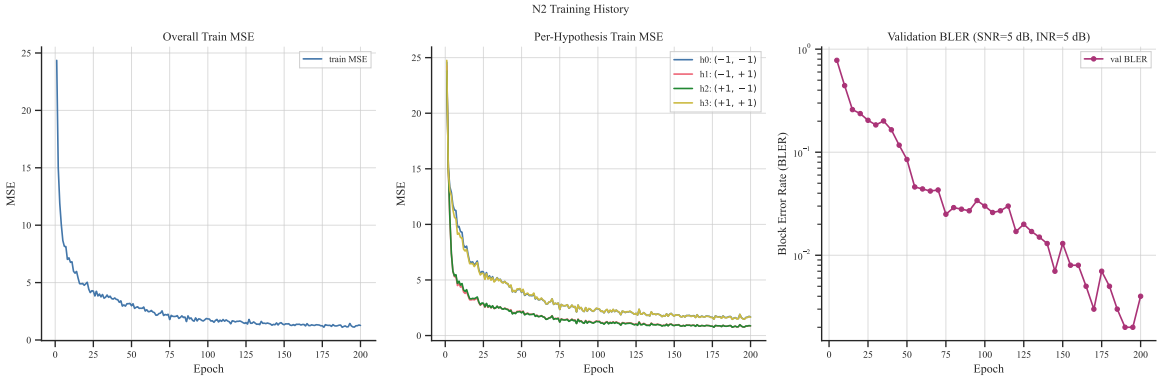
I Training Convergence

Fig. 5 shows the per-epoch training MSE and validation BLER for both seeds; the corresponding summary statistics are given in Table IX.

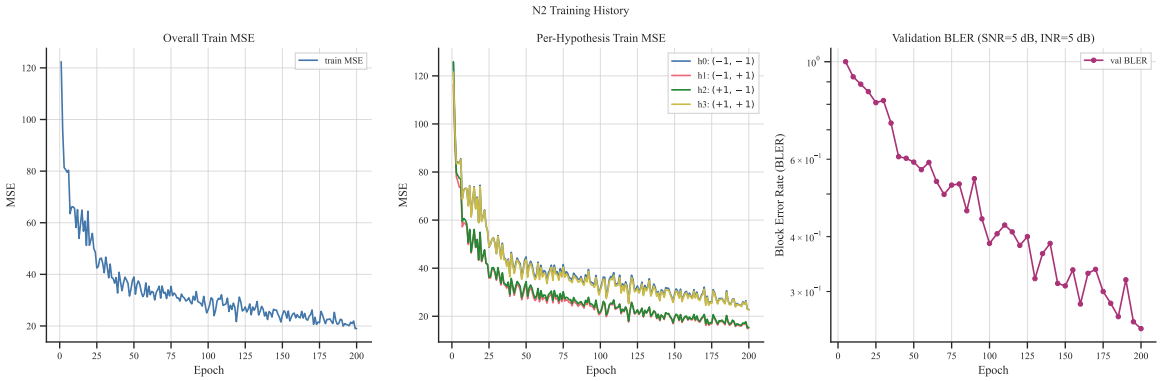


(a) Seed 32 (b) Seed 42

Fig. 4: BLER vs. INR at SNR $= 5$ dB for N2 (both seeds) and baselines. Seed 32 degrades gracefully; B1 saturates above INR ≈ 5 dB. Numerical values in Table VIII.



(a) Seed 32: MSE and validation BLER



(b) Seed 42: MSE and validation BLER

Fig. 5: Per-epoch training MSE and validation BLER for both N2 seeds. Seed 32 converges substantially faster; see Table IX for summary statistics.

Seed 32's much lower final MSE (1.27 vs. 18.97) explains its substantially better BLER: the normalized-target formulation (seed 42) requires the NN to simultaneously learn the oracle distance *and* the $1/\sigma^2$

TABLE IX: Training convergence summary

Seed	Initial MSE	Final MSE	Best Val BLER	Best Epoch
32	24.36	1.27	0.002	190
42	122.4	18.97	0.247	200

scaling, a harder regression problem with a more dispersed target distribution. By externalizing the $1/(2\sigma^2)$ scaling, seed 32 focuses the NN on the interference estimation task.

VI. Comparative Analysis

A Performance at the Operating Point (SNR $= 5$ dB, INR $= 5$ dB)

Table X presents a unified comparison of all methods at the primary operating point.

TABLE X: Summary comparison at SNR $= 5$ dB, INR $= 5$ dB

Method	BLER	FLOPs	Lat. (ms)	% Budget
B2 Oracle Viterbi	1.0×10^{-4}	65,536	0.35	3.1%
N2 seed 32	2.5×10^{-3}	913,664	1981.9	43.5%
B5 IC + Viterbi	3.5×10^{-1}	94,720	1.24	4.5%
N2 seed 42	2.4×10^{-1}	913,664	1614.2	43.5%
B1 Mismatched Viterbi	1.000	65,536	0.30	3.1%

At SNR $= 5$ dB, INR $= 5$ dB, N2 (seed 32) achieves two orders of magnitude lower BLER than both B1 and B5. The SNR gain relative to B1 is estimated at > 5 dB at the BLER $= 10^{-3}$ operating point (B1 requires SNR ≈ 10 dB to reach 10^{-3} in the absence of interference, but never reaches it under INR $= 5$ dB interference; N2 seed 32 reaches 2.5×10^{-3} at SNR $= 5$ dB).

B Seed 32 vs. Seed 42

Both seeds use an identical BiGRU architecture; they differ only in the formulation of the oracle training target:

- **Seed 32** learns unnormalized oracle distances $-\|\mathbf{y}[k] - \mathbf{h}_j - \mathbf{i}[k]\|^2$ and applies the noise scaling $1/(2\sigma^2)$ deterministically at inference. This isolates the NN's learning task to *interference-aware channel estimation*, while noise normalization is handled analytically.
- **Seed 42** learns the full log-likelihood $-\|\mathbf{r} - \mathbf{h}_j - \mathbf{i}\|^2/(2\sigma^2)$, requiring the NN to also capture how the noise variance rescales the metric. The resulting target distribution has a much larger dynamic range (initial MSE $= 122.4$ vs. 24.4 for seed 32), slowing convergence.

C Limitations of the Current Approach

- **Trellis is fixed.** The NASA K=7 code was not designed for sinusoidal interference; the trellis structure may not be optimal. Phase 3 will address this via evolutionary trellis search.
- **Distribution shift.** Training covers $P \in [8, 32]$; performance on $P \notin [8, 32]$ has not been characterized.
- **Single interference model.** Only sinusoidal interference was considered; generalization to other structured interference (e.g., chirp, pulsed) is an open question.

VII. Conclusion and Future Work

This report documents three phases of research toward robust, compute-efficient decoding under sinusoidal interference.

Phase 1 established that classical mismatched Viterbi (B1) saturates at BLER $= 1$ under moderate interference (INR ≥ 5 dB), while the oracle (B2) maintains BLER $= 10^{-4}$ regardless of INR. An FFT-based interference cancellation baseline (B5) provides partial mitigation but degrades sharply above INR ≈ 0 dB due to estimation error.

Phase 2a demonstrated that an end-to-end BiGRU decoder (N1) cannot learn to decode the K=7 code, plateauing near random-guessing accuracy. The root causes are an information bottleneck at the pooling layer and the intractability of implicitly representing a 64-state Viterbi search. N1 also exceeds the 2.1 MFLOP compute budget by $3.17\times$.

Phase 2b introduced the neural branch metric estimator (N2), which preserves the Viterbi dynamic-programming structure and replaces only the branch metric computation. N2 (seed 32) achieves BLER $=$

2.5×10^{-3} at SNR $= 5$ dB, INR $= 5$ dB — a gain exceeding five SNR dB over B1 and within ≈ 1.2 dB of the oracle B2 (by log-linear interpolation of the B2 curve to the matched BLER) — using 43.5% of the 2.1 MFLOP budget. The key insight is that *decomposing* local channel estimation (neural) from global trellis search (classical algorithm) is the critical design principle.

a) Future work (Phase 3).

The trellis itself was fixed to the NASA K=7 code throughout this work. Phase 3 will co-design the trellis via evolutionary search, optimizing the 64-state transition table jointly with the neural branch metric estimator under the sinusoidal interference channel. The hypothesis is that a searched trellis will provide an additional ≥ 1 dB gain over the classical K=7 code when decoded with N2, and that this gain will not be achievable by a random trellis — validating that the search is non-trivial.

References

[1] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Inf. Theory*, vol. 13, no. 2, pp. 260–269, Apr. 1967.

[2] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill, 2008.

[3] J. A. Heller and I. M. Jacobs, "Viterbi decoding for satellite and space communication," *IEEE Trans. Commun. Technol.*, vol. 19, no. 5, pp. 835–848, Oct. 1971.

[4] N. Shlezinger, N. Farsad, Y. C. Eldar, and A. J. Goldsmith, "ViterbiNet: A deep learning based Viterbi algorithm for symbol detection," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3319–3331, May 2020.

[5] T. Gruber, S. Cammerer, J. Hoydis, and S. ten Brink, "On deep learning-based channel decoding," in *Proc. 51st Annu. Conf. Inf. Sci. Syst. (CISS)*, Baltimore, MD, Mar. 2017.

[6] H. Kim, Y. Jiang, R. Rana, S. Kannan, S. Oh, and P. Viswanath, "Communication algorithms via deep learning," in *Proc. Int. Conf. Learning Representations (ICLR)*, Vancouver, Apr. 2018.

[7] T. Kargi and T. M. Duman, "A deep learning based decoder for concatenated coding over deletion channels," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Denver, CO, Jun. 2024.

[8] Z. Yang and Y. Jiang, "Online learning of trellis diagram using neural network," arXiv:2202.10635, Feb. 2022.